HCL Training Project Report

on

StockSmart: Retail Inventory Optimization System

Submitted By

Ananya Agrawal(2400320100187)

B.Tech - Computer Science and Engineering

Under the Supervision of

Sakshi





ABES ENGINEERING COLLEGE, GHAZIABAD

AFFILIATED TO

Dr. A P J ABDUL KALAM TECHNICAL UNIVERSITY, LUCKNOW

September 2026

Table of Contents

1. **Introduction**
    - Project Overview
    - Objectives
2. **Project Description**
    - System Description
    - Features
3. **System Design**
    - System Architecture
    - Tech Stack Used
    - Modules
4. **Functional Requirements**
    - Product Management
    - Supplier Management
    - Location Management
    - Inventory Management
    - REST APIs
5. **Database Design**
    - ER Diagram
6. **Future Enhancements**
7. **Conclusion**

Introduction

1.1 Project Overview

StockSmart: Retail Inventory Optimization System is a backend-based retail inventory management application developed using Java Spring Boot and MySQL. The system provides a centralized platform for managing products, suppliers, retail locations, and inventory quantities.

The application follows a layered architecture consisting of Entity, Repository, Service, and Controller layers. RESTful APIs are provided for performing CRUD operations, while Spring Data JPA and Hibernate are used for database interaction.

A key aspect of StockSmart is its multi-location inventory management, where the quantity of a product is maintained separately for each store or warehouse.

1.2 Objectives

- To develop a centralized system for managing retail products and suppliers.
- To manage inventory across multiple stores and warehouses.
- To maintain accurate location-wise stock quantities.
- To provide RESTful APIs for inventory-related operations.
- To reduce manual effort in managing and tracking inventory data.
- To provide a structured backend that can be extended with inventory optimization features in the future.

Project Description

2.1 System Description

StockSmart: Retail Inventory Optimization System is a backend application designed to manage retail inventory in a centralized and structured manner. The system allows users to manage products, suppliers, locations, and inventory quantities through RESTful APIs.

The system supports inventory management across multiple locations such as stores and warehouses. Inventory is maintained separately for each product and location, allowing the same product to have different stock quantities at different locations.

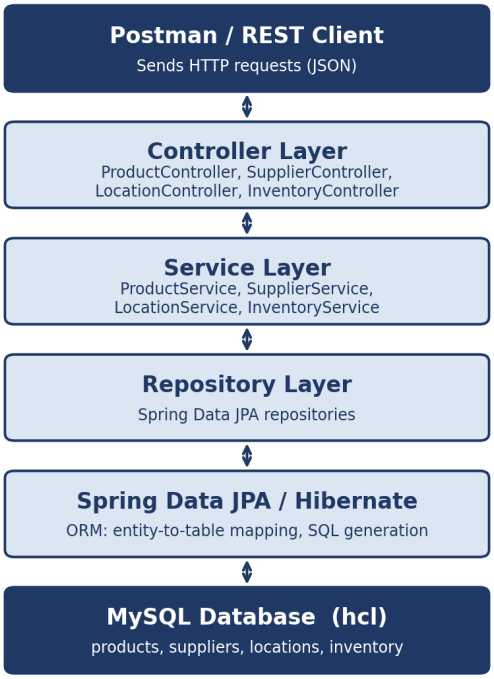
The application is developed using Java Spring Boot and follows a layered architecture with Controller, Service, Repository, and Entity layers. Spring Data JPA and Hibernate are used for database operations, while MySQL is used for persistent data storage.

2.2 Features

- Product Management: Create, view, update, and delete product information.
- Supplier Management: Maintain supplier details and perform CRUD operations.
- Location Management: Manage stores and warehouses along with their details.
- Multi-Location Inventory: Maintain product quantities separately for different locations.
- RESTful APIs: Provide API endpoints for performing system operations.
- Database Persistence: Store and manage application data using MySQL.
- CRUD Operations: Support Create, Read, Update, and Delete operations for major entities.
- Postman Testing: APIs can be tested and verified using Postman.
- Key/Unique Features
- Location-wise Inventory: Stock is associated with both a product and a specific location rather than being stored only at product level.
- Centralized Management: Products, suppliers, locations, and inventory are managed through a single backend system.
- Layered Architecture: Separation of Controller, Service, Repository, and Entity layers improves organization and maintainability.

System Design

3.1 System Architecture



3.2 Tech Stack Used

Technology	Purpose
Java	Core programming language
Spring Boot	Backend application development
Spring Data JPA	Database interaction
Hibernate	ORM and entity mapping
MySQL	Relational database
Maven	Dependency and build management
REST API	Communication between client and backend
Postman	API testing
VS Code	Development environment

3.3 Modules

The system consists of four main modules:

Product Management: Manages product details such as name, SKU, category, price, and reorder level.

Supplier Management: Manages supplier information such as name, email, phone, and address.

Location Management: Manages store and warehouse details.

Inventory Management: Maintains the quantity of each product at a particular location.

Functional Requirements

4.1 Product Management

- The system shall allow users to add new products.
- The system shall allow users to view all products and retrieve a product by ID.
- The system shall allow users to update product details.
- The system shall allow users to delete a product.
- Each product shall contain details such as name, SKU, category, price, and reorder level.

4.2 Supplier Management

- The system shall allow users to add supplier information.
- The system shall allow users to view supplier details.
- The system shall allow users to update supplier information.
- The system shall allow users to delete suppliers.
- Supplier information shall include name, email, phone, and address.

4.3 Location Management

- The system shall allow users to add retail locations.
- The system shall allow users to view, update, and delete locations.
- A location shall represent a store or warehouse.
- Location information shall include name, type, and address.

4.4 Inventory Management

- The system shall allow users to create inventory records for a product at a specific location.
- The system shall allow users to view inventory records.
- The system shall allow users to update stock quantity.
- The system shall allow users to delete inventory records.
- The system shall maintain inventory quantity separately for each product-location combination.

4.5 REST API Requirements

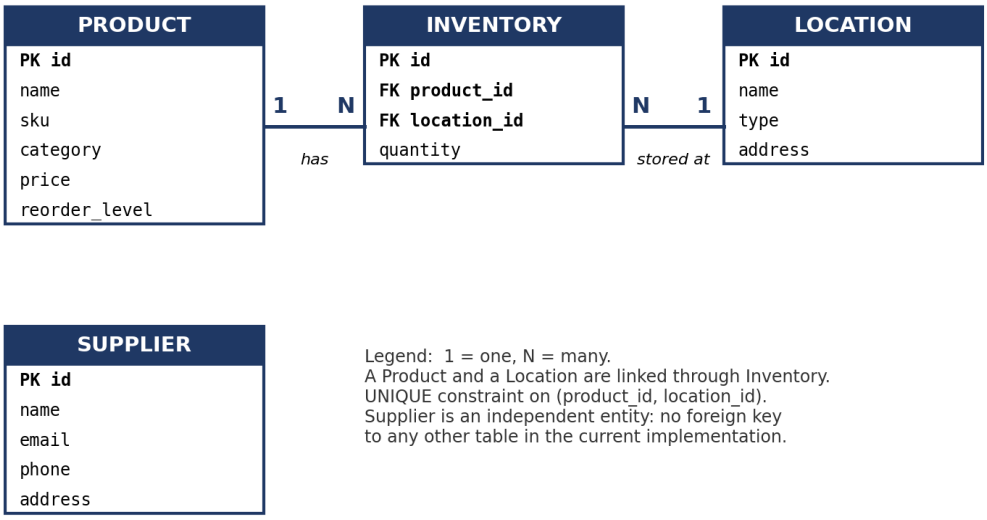
The system shall expose RESTful endpoints for performing the above operations. API requests and responses shall use HTTP and JSON, allowing the backend to be tested through tools such as Postman.

Database Design

5.1 Database Tables

Table	Main Fields	Purpose
Products	id, name, sku, category, price, reorder_level	Stores details of products managed by the system.
Suppliers	id, name, email, phone, address	Stores supplier contact and address information.
Locations	id, name, type, address	Stores details of retail stores and warehouses.
Inventory	id, product_id, location_id, quantity	Stores the quantity of each product available at a particular location.

5.2 ER Diagram



Future Enhancements

- Low-stock alerts to identify products whose quantity falls below the reorder level.
- Automatic reorder management to support purchase and restocking workflows.
- Inventory analytics and reporting for better understanding of stock levels and movement.
- Demand forecasting to help estimate future inventory requirements.
- User authentication and role-based access for secure system access.
- Frontend dashboard for easier visualization and management of inventory data.
- Supplier-product association to manage which suppliers provide specific products.

Conclusion

StockSmart: Retail Inventory Optimization System provides a structured backend solution for managing products, suppliers, locations, and inventory in a retail environment. The system uses Java Spring Boot, Spring Data JPA, Hibernate, and MySQL to provide RESTful APIs and maintain organized inventory data.

The multi-location inventory model allows stock quantities to be maintained for individual products at different stores or warehouses. The modular and layered design also provides a suitable foundation for extending the system with additional inventory management and optimization capabilities in the future.